Panduan Pemasangan & Integrasi

Flutter Frontend ↔ Tanami Backend

Dokumen ini menjelaskan cara menghubungkan aplikasi mobile Flutter ke backend Tanami (FastAPI) yang menyediakan autentikasi, manajemen lahan & tanaman, scan penyakit/kematangan, reminder cuaca, dan forum. Seluruh endpoint, format respons, dan kode galat mengikuti kontrak di **design (1).md** bagian 10 dan TQA bagian 6.

Versi: 1.0 | Tanggal: 27 Oktober 2026 | Backend: tanami-backend (FastAPI + PostgreSQL 16)

1. Prasyarat

- Flutter SDK 3.22+ (Dart 3.4+).
- Backend Tanami sudah berjalan: `uvicorn app.asgi:app --reload --port 8000`
- PostgreSQL 16 + Redis 7 (atau gunakan `pgserver` dari dev dependencies backend).
- (Opsional) tanami-vision berjalan di port 8001 untuk scan end-to-end.
- Backend `.env` sudah diisi (lihat bagian 15).

2. Buat Project Flutter

```
flutter create tanami_app
cd tanami_app
```

Tambahkan dependencies berikut di `pubspec.yaml`:

```
dependencies:
  flutter:
    sdk: flutter
  dio: ^5.6.0
  flutter_riverpod: ^2.5.1
  flutter_secure_storage: ^9.0.0
  shared_preferences: ^2.2.3
  image_picker: ^1.0.7
  image: ^4.2.0
  freezed_annotation: ^2.4.1
  json_annotation: ^4.8.1
  connectivity_plus: ^6.0.5
  uuid: ^4.4.0
  logger: ^2.3.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.9
  freezed: ^2.5.2
  json_serializable: ^6.7.1
  mocktail: ^1.0.3
  integration_test:
    sdk: flutter
```

```
flutter pub get
```

3. Struktur Project yang Disarankan

```
lib/
■■■ main.dart
■■■ config/
■   ■■■ app_config.dart
■■■ core/
■   ■■■ api/
■   ■   ■■■ dio_client.dart
■   ■   ■■■ api_exception.dart
■   ■■■ models/
■   ■   ■■■ envelope.dart
■   ■   ■■■ pagination.dart
■   ■■■ storage/
■       ■■■ secure_storage.dart
■■■ features/
■   ■■■ auth/
■   ■   ■■■ data/auth_repository.dart
■   ■   ■■■ models/auth_models.dart
■   ■   ■■■ providers/auth_provider.dart
```

```
■   ■■■ scans/
■   ■   ■■■ data/scan_repository.dart
■   ■   ■■■ data/offline_queue.dart
■   ■   ■■■ models/scan_models.dart
■   ■■■ farms/
■   ■■■ reminders/
■   ■■■ forum/
■   ■■■ weather/
■■■ shared/
    ■■■ widgets/
    ■■■ utils/image_compress.dart
```

4. Konfigurasi Environment (config/app_config.dart)

```
class AppConfig {
  static const String environment =
      String.fromEnvironment('ENV', defaultValue: 'dev');

  static String get apiBaseUrl {
    switch (environment) {
      case 'prod':    return 'https://api.tanami.id/api/v1';
      case 'staging': return 'https://staging-api.tanami.id/api/v1';
      default:        return 'http://10.0.2.2:8000/api/v1'; // Android emulator
    }
  }
}
```

5. Dio Client + Auth Interceptor

Backend memakai JWT access token (15 menit) + refresh token (30 hari). Interceptor otomatis menyisipkan `Authorization: Bearer` dan mencoba refresh saat 401.

```
class DioClient {
  final Dio dio;
  final FlutterSecureStorage storage;

  DioClient({required this.storage})
      : dio = Dio(BaseOptions(
          baseUrl: AppConfig.apiBaseUrl,
          connectTimeout: const Duration(seconds: 10),
          receiveTimeout: const Duration(seconds: 30),
          headers: {'Accept': 'application/json', 'Accept-Language': 'id'},
        )) {
    dio.interceptors.add(AuthInterceptor(storage));
  }
}

class AuthInterceptor extends Interceptor {
  final FlutterSecureStorage storage;
  AuthInterceptor(this.storage);

  @override
  Future<void> onRequest(RequestOptions o, RequestInterceptorHandler h) async {
    final token = await storage.read(key: 'access_token');
    if (token != null) o.headers['Authorization'] = 'Bearer $token';
    h.next(o);
  }

  @override
  Future<void> onError(DioException err, ErrorInterceptorHandler h) async {
    if (err.response?.statusCode == 401) {
      final refresh = await storage.read(key: 'refresh_token');
      if (refresh != null) {
        try {
          final res = await Dio().post(
            '${AppConfig.apiBaseUrl}/auth/refresh',
```

```
            data: {'refresh_token': refresh},
          );
          final newAccess = res.data['data']['access_token'];
          final newRefresh = res.data['data']['refresh_token'];
          await storage.write(key: 'access_token', value: newAccess);
          await storage.write(key: 'refresh_token', value: newRefresh);

          final retryOpts = err.requestOptions;
          retryOpts.headers['Authorization'] = 'Bearer $newAccess';
          final response = await Dio().fetch(retryOpts);
          h.resolve(response);
          return;
        } catch (_) {
          await storage.deleteAll();
        }
      }
    }
    h.next(err);
  }
}
```

6. Model Envelope & Error Parsing

Backend selalu mengembalikan envelope. Frontend wajib parse envelope, bukan dictionary mentah.

```
class Envelope<T> {
  final T data;
  Envelope({required this.data});
  factory Envelope.fromJson(Map<String, dynamic> json, T Function(Object?) f) {
    return Envelope<T>(data: f(json['data']));
  }
}

class ListEnvelope<T> {
  final List<T> data;
  final PaginationMeta meta;
  ListEnvelope({required this.data, required this.meta});
  factory ListEnvelope.fromJson(Map<String, dynamic> json, T Function(Object?) f) {
    return ListEnvelope<T>(
      data: (json['data'] as List).map((e) => f(e)).toList(),
      meta: PaginationMeta.fromJson(json['meta']),
    );
  }
}

class PaginationMeta {
  final String? nextCursor;
  final bool hasMore;
  PaginationMeta({this.nextCursor, required this.hasMore});
  factory PaginationMeta.fromJson(Map<String, dynamic> json) {
    return PaginationMeta(
      nextCursor: json['next_cursor'] as String?,
      hasMore: json['has_more'] as bool? ?? false,
    );
  }
}

class ApiException implements Exception {
  final int statusCode;
  final String code;
  final String message;
  final Map<String, dynamic>? details;
  final String requestId;

  ApiException({required this.statusCode, required this.code,
    required this.message, this.details, required this.requestId});
```

```
  factory ApiException.fromDioError(DioException e) {
    final body = e.response?.data;
    if (body is Map<String, dynamic> && body['error'] is Map) {
      final err = body['error'] as Map<String, dynamic>;
      return ApiException(
        statusCode: e.response?.statusCode ?? 500,
        code: err['code'] ?? 'internal_error',
        message: err['message'] ?? 'Terjadi kesalahan pada server.',
        details: err['details'] is Map ? Map<String, dynamic>.from(err['details']) : null,
        requestId: err['request_id'] ?? '',
      );
    }
    return ApiException(
      statusCode: e.response?.statusCode ?? 500,
      code: 'internal_error',
      message: e.message ?? '',
      requestId: e.response?.headers.value('x-request-id') ?? '',
    );
  }

  String userMessage() {
    switch (code) {
      case 'unauthorized': return 'Sesi Anda tidak valid atau sudah berakhir.';
      case 'forbidden': return 'Anda tidak memiliki akses ke data ini.';
      case 'not_found': return 'Data tidak ditemukan.';
      case 'conflict': return 'Data sudah ada.';
      case 'payload_too_large': return 'Foto terlalu besar. Kompress ulang.';
      case 'invalid_image': return 'Gambar tidak dapat diproses. Foto ulang.';
      case 'rate_limited': return 'Terlalu banyak permintaan. Coba lagi sebentar.';
      case 'service_unavailable': return 'Layanan sedang tidak tersedia.';
      default: return message.isNotEmpty ? message : 'Terjadi kesalahan pada server.';
    }
  }
}
```

7. Auth Repository & Model

```
class TokenPair {
  final String accessToken;
  final String refreshToken;
  final String tokenType;
  final int expiresIn;
  TokenPair({required this.accessToken, required this.refreshToken,
    required this.tokenType, required this.expiresIn});
  factory TokenPair.fromJson(Map<String, dynamic> json) {
    return TokenPair(
      accessToken: json['access_token'] as String,
      refreshToken: json['refresh_token'] as String,
      tokenType: json['token_type'] as String? ?? 'bearer',
      expiresIn: json['expires_in'] as int,
    );
  }
}

class UserOut {
  final String id;
  final String? email;
  final String? phone;
  final String displayName;
  final String role;
  final bool consentTrainingData;
  final DateTime? consentAt;
  final DateTime createdAt;
  UserOut({required this.id, this.email, this.phone, required this.displayName,
    required this.role, required this.consentTrainingData, this.consentAt,
    required this.createdAt});
  factory UserOut.fromJson(Map<String, dynamic> json) {
```

```
    return UserOut(
      id: json['id'] as String,
      email: json['email'] as String?,
      phone: json['phone'] as String?,
      displayName: json['display_name'] as String,
      role: json['role'] as String,
      consentTrainingData: json['consent_training_data'] as bool? ?? false,
      consentAt: json['consent_at'] != null
          ? DateTime.parse(json['consent_at'] as String) : null,
      createdAt: DateTime.parse(json['created_at'] as String),
    );
  }
}

class AuthRepository {
  final DioClient client;
  AuthRepository(this.client);

  Future<TokenPair> register({String? email, String? phone,
      required String password, required String displayName}) async {
    try {
      final res = await client.dio.post('/auth/register', data: {
        if (email != null) 'email': email,
        if (phone != null) 'phone': phone,
        'password': password,
        'display_name': displayName,
        'consent_training_data': false,
      });
      return TokenPair.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<TokenPair> login({String? email, String? phone, required String password}) async {
    try {
      final res = await client.dio.post('/auth/login', data: {
        if (email != null) 'email': email,
        if (phone != null) 'phone': phone,
        'password': password,
      });
      return TokenPair.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<TokenPair> refresh(String refreshToken) async {
    try {
      final res = await client.dio.post('/auth/refresh',
          data: {'refresh_token': refreshToken});
      return TokenPair.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<void> logout(String refreshToken) async {
    try {
      await client.dio.post('/auth/logout', data: {'refresh_token': refreshToken});
    } on DioException catch (_) {}
  }

  Future<UserOut> me() async {
    try {
      final res = await client.dio.get('/users/me');
      return UserOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
```

```
      throw ApiException.fromDioError(e);
    }
  }
}
```

8. Scan Repository (Multipart + Idempotensi)

Backend mengharuskan `client_uuid` di body dan `Idempotency-Key` di header. Resend dengan uuid sama mengembalikan hasil yang sama tanpa proses ulang.

```
class ScanRepository {
  final DioClient client;
  ScanRepository(this.client);

  Future<ScanCreated> createScan({
    required String scanType, // 'disease' | 'ripeness'
    required File imageFile,
    String? plantId,
    DateTime? capturedAt,
  }) async {
    final clientUuid = const Uuid().v4();
    try {
      final formData = FormData.fromMap({
        'type': scanType,
        'image': await MultipartFile.fromFile(imageFile.path, filename: 'scan.jpg'),
        'client_uuid': clientUuid,
        if (plantId != null) 'plant_id': plantId,
        if (capturedAt != null) 'captured_at': capturedAt.toUtc().toIso8601String(),
      });

      final res = await client.dio.post(
        '/scans',
        data: formData,
        options: Options(headers: {'Idempotency-Key': clientUuid}),
      );
      return ScanCreated.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }
}
```

9. Kompresi Gambar Sebelum Upload

Design §11.1: maks 2 MB, sisi terpanjang ≤ 1280 px. Kompress di Flutter sebelum kirim.

```
Future<File> compressForScan(File original) async {
  final bytes = await original.readAsBytes();
  final image = img.decodeImage(bytes);
  if (image == null) throw Exception('Gambar tidak dapat dibaca');

  img.Image resized = image;
  if (image.width > 1280 || image.height > 1280) {
    resized = img.copyResize(image, width: 1280, height: 1280, maintainAspect: true);
  }

  var jpeg = img.encodeJpg(resized, quality: 88);
  final out = File('${original.path}_compressed.jpg');
  await out.writeAsBytes(jpeg);

  if (await out.length() > 2 * 1024 * 1024) {
    jpeg = img.encodeJpg(resized, quality: 70);
    await out.writeAsBytes(jpeg);
  }
  return out;
}
```

10. Offline Queue untuk Scan

TQA §10: aplikasi tetap bisa dibuka dan mengantre unggahan scan saat jaringan buruk.

```
class OfflineScanQueue {
  static const _key = 'pending_scans';

  Future<void> enqueue(Map<String, dynamic> scanData) async {
    final prefs = await SharedPreferences.getInstance();
    final list = prefs.getStringList(_key) ?? [];
    list.add(jsonEncode(scanData));
    await prefs.setStringList(_key, list);
  }

  Future<List<Map<String, dynamic>>> pending() async {
    final prefs = await SharedPreferences.getInstance();
    final list = prefs.getStringList(_key) ?? [];
    return list.map((e) => jsonDecode(e) as Map<String, dynamic>).toList();
  }

  Future<void> remove(String clientUuid) async {
    final prefs = await SharedPreferences.getInstance();
    final list = prefs.getStringList(_key) ?? [];
    list.removeWhere((e) {
      final m = jsonDecode(e) as Map<String, dynamic>;
      return m['client_uuid'] == clientUuid;
    });
    await prefs.setStringList(_key, list);
  }

  Future<void> syncWhenOnline(ScanRepository repo) async {
    final connectivity = await Connectivity().checkConnectivity();
    if (connectivity == ConnectivityResult.none) return;

    final pending = await this.pending();
    for (final scan in pending) {
      try {
        await repo.createScan(
          scanType: scan['type'],
          imageFile: File(scan['file_path']),
          plantId: scan['plant_id'],
          capturedAt: DateTime.parse(scan['captured_at']),
        );
        await remove(scan['client_uuid']);
      } catch (_) { /* biarkan di queue, coba lagi nanti */ }
    }
  }
}
```

11. Pagination Cursor-Based

Backend pakai cursor-based pagination (`?limit=20&cursor=...`).

```
class PaginatedList<T> extends StatefulWidget {
  final Future<ListEnvelope<T>> Function(String? cursor) fetcher;
  final Widget Function(T item) itemBuilder;
  const PaginatedList({super.key, required this.fetcher, required this.itemBuilder});

  @override
  State<PaginatedList<T>> createState() => _PaginatedListState<T>();
}

class _PaginatedListState<T> extends State<PaginatedList<T>> {
  final List<T> _items = [];
  String? _cursor;
  bool _hasMore = true;
```

```
  bool _loading = false;

  @override
  void initState() { super.initState(); _loadMore(); }

  Future<void> _loadMore() async {
    if (_loading || !_hasMore) return;
    setState(() => _loading = true);
    try {
      final result = await widget.fetcher(_cursor);
      setState(() {
        _items.addAll(result.data);
        _cursor = result.meta.nextCursor;
        _hasMore = result.meta.hasMore;
      });
    } finally { setState(() => _loading = false); }
  }

  @override
  Widget build(BuildContext context) {
    return NotificationListener<ScrollNotification>(
      onNotification: (n) {
        if (n is ScrollEndNotification && n.metrics.extentAfter < 200) _loadMore();
        return false;
      },
      child: ListView.builder(
        itemCount: _items.length + (_hasMore ? 1 : 0),
        itemBuilder: (ctx, i) {
          if (i == _items.length) return const Center(child: CircularProgressIndicator());
          return widget.itemBuilder(_items[i]);
        },
      ),
    );
  }
}
```

12. Error Handling di UI

```
void showApiError(BuildContext context, ApiException e) {
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(content: Text(e.userMessage()), backgroundColor: Colors.red),
  );
}
```

13. Checklist Koneksi Frontend ↔ Backend

Item	Aksi
Base URL	AppConfig.apiBaseUrl sesuai environment (dev/staging/prod).
CORS	Isi TANAMI_CORS_ORIGINS di backend .env.
Token	Simpan di flutter_secure_storage, bukan SharedPreferences.
Auth header	Authorization: Bearer <access_token>.
Refresh	Tangani 401 di interceptor, coba /auth/refresh.
Scan	Kompress ≤ 2MB, kirim Idempotency-Key header + client_uuid form.
Error	Parse envelope {error:{code,message,details,request_id}}, tampilkan userMessage().
Pagination	Gunakan meta.next_cursor dan meta.has_more.
Offline	Antre scan di shared_preferences, sync saat online.
Waktu	Backend kirim ISO 8601 UTC (...Z), konversi ke WIB/WITA/WIT.

BMKG attribution	Tampilkan field attribution di UI cuaca (wajib lisensi BMKG).

14. Troubleshooting Umum

Masalah	Penyebab & Solusi
Connection refused	Emulator Android pakai 10.0.2.2, bukan 127.0.0.1. Device fisik pakai IP LAN.
CORS error	Tambahkan origin Flutter ke TANAMI_CORS_ORIGINS di backend .env.
401 berulang	Token expired, pastikan refresh token tersimpan dan interceptor mencoba /auth/refresh.
Upload 413	Gambar > 2 MB, kompress dulu dengan compressForScan.
Upload 422	Gambar blur/gelap/tidak valid, minta user foto ulang.
Scan tidak muncul di history	Periksa client_uuid unik setiap scan baru; resend dengan uuid sama = duplikat.

15. CORS di Backend (tanami-backend/.env)

```
TANAMI_CORS_ORIGINS=http://localhost:8081,http://10.0.2.2:8081,https://app.tanami.id
TANAMI_JWT_SECRET=<minimal 32 karakter, bukan default dev>
TANAMI_APP_ENV=local   # ganti production saat rilis
```

16. Endpoint Penting (Ringkasan)

Fitur	Endpoint
Auth	POST /auth/register, /auth/login, /auth/refresh, /auth/logout
Profil	GET/PATCH/DELETE /users/me, GET/PATCH /users/me/notification-settings
Device	GET/POST /devices, DELETE /devices/{id}
Farms	GET/POST /farms, GET/PATCH/DELETE /farms/{id}
Plants	GET/POST /farms/{id}/plants, GET/PATCH/DELETE /plants/{id}
Harvest	POST /plants/{id}/harvests
Timeline/Stats	GET /plants/{id}/timeline, /plants/{id}/stats
Scan	POST /scans, GET /scans, GET /scans/{id}, DELETE /scans/{id}, PATCH /scans/{id}, PUT /scans/{id}/fee
Weather	GET /farms/{id}/weather
Reminders	GET /reminders, GET /reminders/{id}, PATCH /reminders/{id}, GET /home
Diseases	GET /diseases, GET /diseases/{code}
Forum	GET/POST /forum/posts, GET/DELETE /forum/posts/{id}, comments, reports
Regions	GET /regions/search?q=..., GET /regions/resolve
Upload	POST /uploads/forum-image

17. Layar Scan Lengkap (Kamera → Upload → Hasil)

Contoh implementasi satu layar scan end-to-end: ambil foto, kompress, upload, tampilkan hasil diagnosis. Menggunakan `image_picker`, `ScanRepository`, dan Riverpod.

```
class ScanScreen extends ConsumerStatefulWidget {
  const ScanScreen({super.key});

  @override
  ConsumerState<ScanScreen> createState() => _ScanScreenState();
}

class _ScanScreenState extends ConsumerState<ScanScreen> {
  File? _imageFile;
  bool _loading = false;
  ScanCreated? _result;
  String? _error;

  Future<void> _pickImage(ImageSource source) async {
    final picker = ImagePicker();
    final picked = await picker.pickImage(source: source, imageQuality: 90);
    if (picked == null) return;

    final compressed = await compressForScan(File(picked.path));
    setState(() {
      _imageFile = compressed;
      _result = null;
      _error = null;
    });
  }

  Future<void> _uploadScan() async {
    if (_imageFile == null) return;
    setState(() { _loading = true; _error = null; });

    try {
      final repo = ref.read(scanRepositoryProvider);
      final result = await repo.createScan(
        scanType: 'disease', // atau 'ripeness'
        imageFile: _imageFile!,
        plantId: null, // isi jika scan dikaitkan ke tanaman
      );
      setState(() { _result = result; });
    } on ApiException catch (e) {
      setState(() { _error = e.userMessage(); });
    } finally {
      setState(() { _loading = false; });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Scan Daun')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            // --- Pilih foto ---
            if (_imageFile == null)
              Row(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: [
                  ElevatedButton.icon(
                    onPressed: () => _pickImage(ImageSource.camera),
                    icon: const Icon(Icons.camera_alt),
```

```
                label: const Text('Kamera'),
              ),
              ElevatedButton.icon(
                onPressed: () => _pickImage(ImageSource.gallery),
                icon: const Icon(Icons.photo_library),
                label: const Text('Galeri'),
              ),
            ],
          ),

          // --- Preview foto ---
          if (_imageFile != null) ...[
            Image.file(_imageFile!, height: 250, fit: BoxFit.cover),
            const SizedBox(height: 12),
            ElevatedButton(
              onPressed: _loading ? null : _uploadScan,
              child: _loading
                  ? const SizedBox(
                      height: 20, width: 20,
                      child: CircularProgressIndicator(strokeWidth: 2),
                    )
                  : const Text('Analisis'),
            ),
          ],

          // --- Error ---
          if (_error != null) ...[
            const SizedBox(height: 12),
            Text(_error!, style: const TextStyle(color: Colors.red)),
          ],

          // --- Hasil ---
          if (_result != null) ...[
            const SizedBox(height: 16),
            _ScanResultCard(result: _result!),
          ],
        ],
      ),
    ),
  );
  }
}
```

Kartu hasil scan yang menampilkan status, diagnosis, dan rekomendasi dari tabel `diseases` (bukan dari model).

```
class _ScanResultCard extends StatelessWidget {
  final ScanCreated result;
  const _ScanResultCard({required this.result});

  @override
  Widget build(BuildContext context) {
    final top = result.result['top'] as Map<String, dynamic>?;
    final advice = result.result['advice'] as Map<String, dynamic>?;
    final quality = result.result['quality'] as Map<String, dynamic>?;

    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            // Status badge
            Row(
              children: [
                _StatusBadge(status: result.status),
                const Spacer(),
                Text(result.modelVersion, style: const TextStyle(fontSize: 10, color: Colors.grey)),
```

```
              ],
            ),
            const SizedBox(height: 12),

            // Diagnosis
            if (top != null) ...[
              Text(
                top['label'] as String? ?? 'Tidak teridentifikasi',
                style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 4),
              Text(
                'Keyakinan: ${((top['confidence'] as num?)?.toDouble() ?? 0) * 100}%',
                style: const TextStyle(color: Colors.grey),
              ),
              const SizedBox(height: 8),
              // Alternatif
              if (result.result['alternatives'] is List) ...[
                const Text('Kemungkinan lain:', style: TextStyle(fontWeight: FontWeight.w600)),
                ...((result.result['alternatives'] as List).map((alt) {
                  final a = alt as Map<String, dynamic>;
                  return Text('  • ${a['label']} (${((a['confidence'] as num?)?.toDouble() ?? 0) * 100}%
                })),
              ],
            ] else
              const Text('Belum bisa dipastikan. Coba foto ulang.', style: TextStyle(color: Colors.orang

            // Advice dari knowledge base
            if (advice != null) ...[
              const Divider(),
              const Text('Saran:', style: TextStyle(fontWeight: FontWeight.w600)),
              Text(advice['summary'] as String? ?? ''),
              if (advice['disease_detail_url'] != null)
                TextButton(
                  onPressed: () { /* navigate ke /diseases/{code} */ },
                  child: const Text('Lihat detail penyakit'),
                ),
            ],

            // Quality report
            if (quality != null) ...[
              const Divider(),
              const Text('Kualitas foto:', style: TextStyle(fontWeight: FontWeight.w600)),
              ...quality.entries.map((e) => Text('  ${e.key}: ${e.value}')),
            ],
          ],
        ),
      ),
    );
  }
}

class _StatusBadge extends StatelessWidget {
  final String status;
  const _StatusBadge({required this.status});

  @override
  Widget build(BuildContext context) {
    final color = switch (status) {
      'confident' => Colors.green,
      'uncertain' => Colors.orange,
      'unknown' => Colors.grey,
      'invalid_image' => Colors.red,
      _ => Colors.grey,
    };
    final label = switch (status) {
      'confident' => 'Yakin',
      'uncertain' => 'Belum yakin',
```

```
      'unknown' => 'Tidak diketahui',
      'invalid_image' => 'Gambar tidak valid',
      _ => status,
    };
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
      decoration: BoxDecoration(
        color: color.withOpacity(0.15),
        borderRadius: BorderRadius.circular(8),
        border: Border.all(color: color),
      ),
      child: Text(label, style: TextStyle(color: color, fontWeight: FontWeight.w600)),
    );
  }
}
```

Penjelasan alur:

- **Ambil foto** via `ImagePicker` (kamera atau galeri).
- **Kompress** dengan `compressForScan` (≤ 2 MB, ≤ 1280 px).
- **Upload** via `ScanRepository.createScan` (multipart + Idempotency-Key).
- **Tampilkan hasil**: status badge, diagnosis top, alternatif, advice dari knowledge base, dan quality report.
- **Handling error**: 413 (foto terlalu besar), 422 (gambar tidak valid), 429 (rate limit), 503 (service unavailable).
- **Idempotensi**: jika user tekan “Analisis” dua kali, `is_duplicate` akan true pada percobaan kedua dan hasil yang sama dikembalikan tanpa proses ulang.

18. Setup main.dart (ProviderScope + GoRouter)

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: TanamiApp()));
}

class TanamiApp extends ConsumerWidget {
  const TanamiApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final authState = ref.watch(authStateProvider);

    return MaterialApp.router(
      title: 'Tanami',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF2E7D32)),
        useMaterial3: true,
      ),
      routerConfig: _router(authState),
    );
  }
}

GoRouter _router(AsyncValue<UserOut?> authState) {
  return GoRouter(
    initialLocation: '/login',
    redirect: (context, state) {
      final isLoggedIn = authState.valueOrNull != null;
      final isLoginRoute = state.matchedLocation == '/login' ||
          state.matchedLocation == '/register';
```

```
      if (!isLoggedIn && !isLoginRoute) return '/login';
      if (isLoggedIn && isLoginRoute) return '/home';
      return null;
    },
    routes: [
      GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),
      GoRoute(path: '/register', builder: (_, __) => const RegisterScreen()),
      GoRoute(path: '/home', builder: (_, __) => const HomeScreen()),
      GoRoute(path: '/scan', builder: (_, __) => const ScanScreen()),
      GoRoute(path: '/farms', builder: (_, __) => const FarmListScreen()),
      GoRoute(path: '/reminders', builder: (_, __) => const ReminderListScreen()),
      GoRoute(path: '/forum', builder: (_, __) => const ForumScreen()),
    ],
  );
}
```

Dependensi tambahan untuk routing:

```
dependencies:
  go_router: ^14.2.0
```

19. Provider Setup (Riverpod)

```
// features/scans/providers/scan_provider.dart
final scanRepositoryProvider = Provider<ScanRepository>((ref) {
  return ScanRepository(ref.watch(dioClientProvider));
});

// features/farms/providers/farm_provider.dart
final farmRepositoryProvider = Provider<FarmRepository>((ref) {
  return FarmRepository(ref.watch(dioClientProvider));
});

// features/reminders/providers/reminder_provider.dart
final reminderRepositoryProvider = Provider<ReminderRepository>((ref) {
  return ReminderRepository(ref.watch(dioClientProvider));
});

// features/weather/providers/weather_provider.dart
final weatherRepositoryProvider = Provider<WeatherRepository>((ref) {
  return WeatherRepository(ref.watch(dioClientProvider));
});
```

20. Farm & Plant Model + Repository

```
class FarmOut {
  final String id;
  final String name;
  final String adm4Code;
  final double? lat;
  final double? lon;
  final double? areaM2;
  final int plantCount;
  final DateTime? archivedAt;
  final DateTime createdAt;
  final DateTime updatedAt;

  FarmOut({required this.id, required this.name, required this.adm4Code,
    this.lat, this.lon, this.areaM2, required this.plantCount,
    this.archivedAt, required this.createdAt, required this.updatedAt});

  factory FarmOut.fromJson(Map<String, dynamic> json) {
    return FarmOut(
      id: json['id'] as String,
      name: json['name'] as String,
      adm4Code: json['adm4_code'] as String,
```

```
      lat: (json['lat'] as num?)?.toDouble(),
      lon: (json['lon'] as num?)?.toDouble(),
      areaM2: (json['area_m2'] as num?)?.toDouble(),
      plantCount: json['plant_count'] as int? ?? 0,
      archivedAt: json['archived_at'] != null
          ? DateTime.parse(json['archived_at'] as String) : null,
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }
}

class PlantOut {
  final String id;
  final String farmId;
  final String label;
  final String commodityCode;
  final String? variety;
  final DateTime plantedAt;
  final String growthStage;
  final String healthStatus;
  final String? healthStatusReason;
  final DateTime? expectedHarvestMin;
  final DateTime? expectedHarvestMax;

  PlantOut({required this.id, required this.farmId, required this.label,
    required this.commodityCode, this.variety, required this.plantedAt,
    required this.growthStage, required this.healthStatus,
    this.healthStatusReason, this.expectedHarvestMin, this.expectedHarvestMax});

  factory PlantOut.fromJson(Map<String, dynamic> json) {
    return PlantOut(
      id: json['id'] as String,
      farmId: json['farm_id'] as String,
      label: json['label'] as String,
      commodityCode: json['commodity_code'] as String,
      variety: json['variety'] as String?,
      plantedAt: DateTime.parse(json['planted_at'] as String),
      growthStage: json['growth_stage'] as String,
      healthStatus: json['health_status'] as String,
      healthStatusReason: json['health_status_reason'] as String?,
      expectedHarvestMin: json['expected_harvest_min'] != null
          ? DateTime.parse(json['expected_harvest_min'] as String) : null,
      expectedHarvestMax: json['expected_harvest_max'] != null
          ? DateTime.parse(json['expected_harvest_max'] as String) : null,
    );
  }
}

class FarmRepository {
  final DioClient client;
  FarmRepository(this.client);

  Future<ListEnvelope<FarmOut>> listFarms({bool includeArchived = false}) async {
    try {
      final res = await client.dio.get('/farms', queryParameters: {
        'include_archived': includeArchived,
      });
      return ListEnvelope<FarmOut>.fromJson(
        res.data as Map<String, dynamic>,
        (json) => FarmOut.fromJson(json as Map<String, dynamic>),
      );
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<FarmOut> createFarm({
```

```
    required String name,
    required String adm4Code,
    double? lat,
    double? lon,
    double? areaM2,
  }) async {
    try {
      final res = await client.dio.post('/farms', data: {
        'name': name,
        'adm4_code': adm4Code,
        if (lat != null) 'lat': lat,
        if (lon != null) 'lon': lon,
        if (areaM2 != null) 'area_m2': areaM2,
      });
      return FarmOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<List<PlantOut>> listPlants(String farmId, {bool includeArchived = false}) async {
    try {
      final res = await client.dio.get('/farms/$farmId/plants', queryParameters: {
        'include_archived': includeArchived,
      });
      final envelope = ListEnvelope<PlantOut>.fromJson(
        res.data as Map<String, dynamic>,
        (json) => PlantOut.fromJson(json as Map<String, dynamic>),
      );
      return envelope.data;
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<PlantOut> createPlant({
    required String farmId,
    required String label,
    required String commodityCode,
    required DateTime plantedAt,
    String? variety,
  }) async {
    try {
      final res = await client.dio.post('/farms/$farmId/plants', data: {
        'label': label,
        'commodity_code': commodityCode,
        'planted_at': plantedAt.toUtc().toIso8601String().split('T').first,
        if (variety != null) 'variety': variety,
      });
      return PlantOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }
}
```

21. Weather Model & Repository

```
class WeatherOut {
  final String adm4Code;
  final String source;
  final DateTime fetchedAt;
  final String attribution;
  final bool isStale;
  final List<WeatherDayOut> days;

  WeatherOut({required this.adm4Code, required this.source,
```

```
    required this.fetchedAt, required this.attribution,
    required this.isStale, required this.days});

  factory WeatherOut.fromJson(Map<String, dynamic> json) {
    return WeatherOut(
      adm4Code: json['adm4_code'] as String,
      source: json['source'] as String,
      fetchedAt: DateTime.parse(json['fetched_at'] as String),
      attribution: json['attribution'] as String,
      isStale: json['is_stale'] as bool? ?? false,
      days: (json['days'] as List)
          .map((e) => WeatherDayOut.fromJson(e as Map<String, dynamic>))
          .toList(),
    );
  }
}

class WeatherDayOut {
  final DateTime date;
  final List<WeatherSlotOut> slots;
  WeatherDayOut({required this.date, required this.slots});
  factory WeatherDayOut.fromJson(Map<String, dynamic> json) {
    return WeatherDayOut(
      date: DateTime.parse(json['date'] as String),
      slots: (json['slots'] as List)
          .map((e) => WeatherSlotOut.fromJson(e as Map<String, dynamic>))
          .toList(),
    );
  }
}

class WeatherSlotOut {
  final DateTime localDatetime;
  final double? t;       // temperature_c
  final int? hu;         // humidity_pct
  final String? weatherDesc;
  final double? ws;      // wind_speed
  final String? wd;      // wind_direction

  WeatherSlotOut({required this.localDatetime, this.t, this.hu,
    this.weatherDesc, this.ws, this.wd});

  factory WeatherSlotOut.fromJson(Map<String, dynamic> json) {
    return WeatherSlotOut(
      localDatetime: DateTime.parse(json['local_datetime'] as String),
      t: (json['t'] as num?)?.toDouble(),
      hu: json['hu'] as int?,
      weatherDesc: json['weather_desc'] as String?,
      ws: (json['ws'] as num?)?.toDouble(),
      wd: json['wd'] as String?,
    );
  }
}

class WeatherRepository {
  final DioClient client;
  WeatherRepository(this.client);

  Future<WeatherOut> getFarmWeather(String farmId) async {
    try {
      final res = await client.dio.get('/farms/$farmId/weather');
      return WeatherOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }
}
```

22. Reminder Model & Repository

```
enum ReminderType { fungalRisk, heavyRainHarvest, harvestSoon, heatStress, scanFollowup, general }
enum ReminderPriority { low, normal, high }
enum ReminderStatus { unread, read, done, dismissed }

class ReminderOut {
  final String id;
  final ReminderType type;
  final ReminderPriority priority;
  final ReminderStatus status;
  final String title;
  final String body;
  final Map<String, dynamic> payload;
  final String? plantId;
  final String? farmId;
  final DateTime? validUntil;
  final DateTime? sentAt;
  final DateTime? readAt;
  final DateTime createdAt;

  ReminderOut({required this.id, required this.type, required this.priority,
    required this.status, required this.title, required this.body,
    required this.payload, this.plantId, this.farmId, this.validUntil,
    this.sentAt, this.readAt, required this.createdAt});

  factory ReminderOut.fromJson(Map<String, dynamic> json) {
    return ReminderOut(
      id: json['id'] as String,
      type: _parseReminderType(json['type'] as String),
      priority: _parseReminderPriority(json['priority'] as String),
      status: _parseReminderStatus(json['status'] as String),
      title: json['title'] as String,
      body: json['body'] as String,
      payload: Map<String, dynamic>.from(json['payload'] as Map? ?? {}),
      plantId: json['plant_id'] as String?,
      farmId: json['farm_id'] as String?,
      validUntil: json['valid_until'] != null
          ? DateTime.parse(json['valid_until'] as String) : null,
      sentAt: json['sent_at'] != null
          ? DateTime.parse(json['sent_at'] as String) : null,
      readAt: json['read_at'] != null
          ? DateTime.parse(json['read_at'] as String) : null,
      createdAt: DateTime.parse(json['created_at'] as String),
    );
  }
}

ReminderType _parseReminderType(String v) =>
    ReminderType.values.firstWhere((e) => e.name == _camelCase(v), orElse: () => ReminderType.general);
ReminderPriority _parseReminderPriority(String v) =>
    ReminderPriority.values.firstWhere((e) => e.name == _camelCase(v), orElse: () => ReminderPriority.no
ReminderStatus _parseReminderStatus(String v) =>
    ReminderStatus.values.firstWhere((e) => e.name == _camelCase(v), orElse: () => ReminderStatus.unread
String _camelCase(String snake) {
  final parts = snake.split('_');
  return parts.first + parts.skip(1).map((p) => p[0].toUpperCase() + p.substring(1)).join();
}

class ReminderRepository {
  final DioClient client;
  ReminderRepository(this.client);

  Future<ListEnvelope<ReminderOut>> listReminders({
    String? status,
    int limit = 20,
    String? cursor,
```

```
  }) async {
    try {
      final res = await client.dio.get('/reminders', queryParameters: {
        if (status != null) 'status': status,
        'limit': limit,
        if (cursor != null) 'cursor': cursor,
      });
      return ListEnvelope<ReminderOut>.fromJson(
        res.data as Map<String, dynamic>,
        (json) => ReminderOut.fromJson(json as Map<String, dynamic>),
      );
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<ReminderOut> getReminder(String id) async {
    try {
      final res = await client.dio.get('/reminders/$id');
      return ReminderOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<ReminderOut> updateReminder(String id, {ReminderStatus? status}) async {
    try {
      final res = await client.dio.patch('/reminders/$id', data: {
        if (status != null) 'status': status.name,
      });
      return ReminderOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }
}
```

23. Forum Model & Repository

```
class ForumPostOut {
  final String id;
  final String userId;
  final String commodityCode;
  final String title;
  final String body;
  final List<String> imageKeys;
  final String? scanId;
  final String status;
  final int commentCount;
  final DateTime createdAt;
  final DateTime updatedAt;

  ForumPostOut({required this.id, required this.userId, required this.commodityCode,
    required this.title, required this.body, required this.imageKeys,
    this.scanId, required this.status, required this.commentCount,
    required this.createdAt, required this.updatedAt});

  factory ForumPostOut.fromJson(Map<String, dynamic> json) {
    return ForumPostOut(
      id: json['id'] as String,
      userId: json['user_id'] as String,
      commodityCode: json['commodity_code'] as String,
      title: json['title'] as String,
      body: json['body'] as String,
      imageKeys: (json['image_keys'] as List?)?.cast<String>() ?? [],
      scanId: json['scan_id'] as String?,
      status: json['status'] as String,
```

```
      commentCount: json['comment_count'] as int? ?? 0,
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }
}

class ForumCommentOut {
  final String id;
  final String postId;
  final String userId;
  final String body;
  final String status;
  final DateTime createdAt;
  final DateTime updatedAt;

  ForumCommentOut({required this.id, required this.postId, required this.userId,
    required this.body, required this.status, required this.createdAt,
    required this.updatedAt});

  factory ForumCommentOut.fromJson(Map<String, dynamic> json) {
    return ForumCommentOut(
      id: json['id'] as String,
      postId: json['post_id'] as String,
      userId: json['user_id'] as String,
      body: json['body'] as String,
      status: json['status'] as String,
      createdAt: DateTime.parse(json['created_at'] as String),
      updatedAt: DateTime.parse(json['updated_at'] as String),
    );
  }
}

class ForumRepository {
  final DioClient client;
  ForumRepository(this.client);

  Future<ListEnvelope<ForumPostOut>> listPosts({
    String? commodityCode,
    String? query,
    int limit = 20,
    String? cursor,
  }) async {
    try {
      final res = await client.dio.get('/forum/posts', queryParameters: {
        if (commodityCode != null) 'commodity_code': commodityCode,
        if (query != null) 'q': query,
        'limit': limit,
        if (cursor != null) 'cursor': cursor,
      });
      return ListEnvelope<ForumPostOut>.fromJson(
        res.data as Map<String, dynamic>,
        (json) => ForumPostOut.fromJson(json as Map<String, dynamic>),
      );
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<ForumPostOut> createPost({
    required String commodityCode,
    required String title,
    required String body,
    String? scanId,
    List<String> imageKeys = const [],
  }) async {
    try {
      final res = await client.dio.post('/forum/posts', data: {
```

```
        'commodity_code': commodityCode,
        'title': title,
        'body': body,
        if (scanId != null) 'scan_id': scanId,
        'image_keys': imageKeys,
      });
      return ForumPostOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<List<ForumCommentOut>> listComments(String postId) async {
    try {
      final res = await client.dio.get('/forum/posts/$postId/comments');
      final envelope = ListEnvelope<ForumCommentOut>.fromJson(
        res.data as Map<String, dynamic>,
        (json) => ForumCommentOut.fromJson(json as Map<String, dynamic>),
      );
      return envelope.data;
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<ForumCommentOut> createComment(String postId, String body) async {
    try {
      final res = await client.dio.post('/forum/posts/$postId/comments', data: {
        'body': body,
      });
      return ForumCommentOut.fromJson(res.data['data'] as Map<String, dynamic>);
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }

  Future<void> reportContent({
    required String targetType, // 'post' | 'comment'
    required String targetId,
    required String reason, // 'spam' | 'scam' | 'abuse' | 'wrong_info' | 'other'
    String? note,
  }) async {
    try {
      await client.dio.post('/forum/reports', data: {
        'target_type': targetType,
        'target_id': targetId,
        'reason': reason,
        if (note != null) 'note': note,
      });
    } on DioException catch (e) {
      throw ApiException.fromDioError(e);
    }
  }
}
```

24. Testing (Unit + Integration)

Unit test repository dengan mock Dio:

```
class MockDio extends Mock implements Dio {}

void main() {
  late MockDio mockDio;
  late AuthRepository repo;

  setUp(() {
    mockDio = MockDio();
```

```
    final client = DioClient(storage: const FlutterSecureStorage());
    // inject mock
    repo = AuthRepository(client);
  });

  test('login returns TokenPair', () async {
    when(() => mockDio.post('/auth/login', data: any(named: 'data')))
        .thenAnswer((_) async => Response(
              data: {'data': {'access_token': 'acc', 'refresh_token': 'ref',
                'token_type': 'bearer', 'expires_in': 900}},
              statusCode: 200,
              requestOptions: RequestOptions(path: '/auth/login'),
            ));
    // final tokens = await repo.login(email: 'a@b.com', password: 'secret');
    // expect(tokens.accessToken, 'acc');
  });
}
```

Integration test (backend nyata harus hidup):

```
void main() {
  final client = DioClient(storage: const FlutterSecureStorage());
  final repo = AuthRepository(client);

  test('register → login → me → logout', () async {
    final email = 'test-${DateTime.now().millisecondsSinceEpoch}@example.com';
    final tokens = await repo.register(
      email: email, password: 'TestPass123!', displayName: 'Test User',
    );
    expect(tokens.accessToken, isNotEmpty);

    final loginTokens = await repo.login(email: email, password: 'TestPass123!');
    expect(loginTokens.accessToken, isNotEmpty);

    final user = await repo.me();
    expect(user.email, email);

    await repo.logout(loginTokens.refreshToken);
  });
}
```

Jalankan:

```
flutter test                    # unit tests
flutter test integration_test/  # integration (backend harus hidup)
```

25. Environment & Build

```
# Dev (emulator Android)
flutter run --dart-define=ENV=dev

# Staging
flutter run --dart-define=ENV=staging

# Production build
flutter build apk --dart-define=ENV=prod
flutter build ios --dart-define=ENV=prod
```

Catatan penting:

- Android emulator mengakses localhost host via `10.0.2.2`.
- iOS simulator mengakses via `127.0.0.1`.
- Device fisik mengakses via IP LAN komputer (mis. `192.168.1.5`).
- Backend CORS harus diisi dengan origin yang sesuai (lihat bagian 15).

26. Ringkasan Model & Repository yang Tersedia

Model	File	Repository
TokenPair, UserOut	auth_models.dart	AuthRepository
ScanCreated, ScanOut, ScanDetailOut	scan_models.dart	ScanRepository
FarmOut, PlantOut	farm_models.dart	FarmRepository
WeatherOut, WeatherDayOut, WeatherSlotOut	weather_models.dart	WeatherRepository
ReminderOut	reminder_models.dart	ReminderRepository
ForumPostOut, ForumCommentOut	forum_models.dart	ForumRepository
Envelope, ListEnvelope, PaginationMeta	envelope.dart	—
ApiException	api_exception.dart	—

Dokumen ini dibuat dari kontrak yang ada di `design (1).md` bagian 10 dan TQA bagian 6. Untuk detail endpoint, buka `http://127.0.0.1:8000/docs` (Swagger UI) saat backend berjalan.

Dokumen ini dibuat dari kontrak yang ada di `design (1).md` bagian 10 dan TQA bagian 6. Untuk detail endpoint, buka `http://127.0.0.1:8000/docs` (Swagger UI) saat backend berjalan.